

Evolution of H-Net Tokenization During Training

Comparing 1-Stage vs 2-Stage Architecture on Polymer SMILES (PI1M)

Example PSMILES: *C(=O)c1ccc(C(=O)Nc2ccc(C(*)=O)cc2N)cc1

Early (1M bytes)

1-Stage:	<chem>* C (= O) c 1 c c (C (= O) N c 2 c c (C (*) = O) cc 2 N) cc 1</chem>	[22 tok, 15 uniq, avg: 1.8]
2-Stage:	<chem>*C(= O) c 1 c c (C (= O) N c 2 c c (C (*) = O) cc 2 N) cc 1</chem>	[27 tok, 15 uniq, avg: 1.4]

Mid (100M bytes)

1-Stage:	<chem>*C(=O) c 1 c c (C(=O)N c 2 c c (C(*) =O)c c 2N)c c 1</chem>	[15 tok, 12 uniq, avg: 2.6]
2-Stage:	<chem>*C(=O) c 1 c c (C(=O) N c 2 c c (C(*) =O)c c 2N)cc 1</chem>	[21 tok, 14 uniq, avg: 1.9]

Converged (238M bytes)

1-Stage:	<chem>*C(=O) c 1 c c (C(=O)N c2ccc (C(*) =O)c c 2N) c c 1</chem>	[16 tok, 13 uniq, avg: 2.4]
2-Stage:	<chem>*C(= O) c 1 c c (C(=O)N c 2 c c (C(*)=O)c c2N) c c 1</chem>	[23 tok, 14 uniq, avg: 1.7]

Key Observations:

- Early training: Both models produce near byte-level tokenization (short tokens)
- Mid training: 1-stage model learns to group characters into chemical subunits (e.g., 'C(=O)N', 'ccc')
- Converged: Token patterns stabilize; 1-stage shows stronger chemical grouping
- The 1-stage architecture achieves better compression (fewer tokens, higher avg length)

Legend: | = token boundary